Universal .zip Deployment Guide

This guide shows **end-to-end, copy-pasteable steps** to deploy an application that you currently have as a **.zip archive** across the most common targets:

- **Local/Server**: Web servers (Nginx/Apache), Application servers (Tomcat/JBoss/WildFly)
- **Containers**: Docker
- **Orchestrator**: Kubernetes
- **AWS**: Lambda, Elastic Beanstalk, CodeDeploy, EC2, S3 static site
- **Azure**: App Service, Functions, VM, Static Website on Storage
- **GCP**: Cloud Functions, Cloud Run, App Engine, Cloud Storage static site

A .zip is just a **container for files**. Most platforms don't *run* zip files; they expect source/binaries in a specific shape. Some services **accept .zip directly** and auto-extract (e.g., Lambda, Azure App Service/Functions, AWS CodeDeploy, Elastic Beanstalk, GCP Cloud Functions). Everywhere else, you'll **unzip and deploy the contents** (or bake them into a Docker image).

---

0) Prerequisites (applies to all)

1. **Have a clean zip layout** (example):
2. myapp.zip
3. ├─ bin/ # startup scripts (start.sh, run.ps1, etc.)
4. ├─ app/ # app code (src, compiled JAR, Node build, etc.)
5. ├─ public/ # static assets (if any)
6. ├─ config/ # config files, env templates
7. └─ README.md
8. **Decide your start command** (examples):
    - Java JAR: java -jar app/myapp.jar
    - Node: node app/server.js or npm start
    - Python: gunicorn app.wsgi:application --bind 0.0.0.0:8000
    - Static: serve via Nginx/Apache (no command)
9. **Create a .env file** if needed and a simple **health endpoint** (/healthz).
10. Install CLIs you need: docker, kubectl, aws, az, gcloud, zip, unzip.

---

1) Deploy to a Web Server (Nginx/Apache)

Linux server (Ubuntu/Debian) with Nginx

```
sudo apt-get update && sudo apt-get install -y nginx unzip
sudo rm -rf /var/www/html/*
sudo unzip myapp.zip -d /var/www/html
# Optional: point root to extracted folder if nested
# sudo nano /etc/nginx/sites-available/default
#   root /var/www/html/public;  # if your index.html lives there
sudo nginx -t && sudo systemctl reload nginx
```

Apache (httpd)

```
sudo apt-get update && sudo apt-get install -y apache2 unzip
sudo rm -rf /var/www/html/*
sudo unzip myapp.zip -d /var/www/html
sudo systemctl reload apache2
```

Use this for **static sites** or if your app is served behind Nginx/Apache as a reverse proxy.

---

2) Deploy to an Application Server (Tomcat / JBoss / WildFly)

If your app is a Java web app, convert to **WAR** (a WAR is a zip with a specific layout):

Create a WAR from your zip (simple pattern)

```
unzip myapp.zip -d extracted
# Ensure Java webapp layout under extracted/: WEB-INF/, classes/, lib/, web.xml, etc.
cd extracted && zip -r ../myapp.war .
```

Deploy to Tomcat

```
sudo cp myapp.war /opt/tomcat/webapps/
# Tomcat auto-expands and deploys it. Check logs:
sudo tail -f /opt/tomcat/logs/catalina.out
```

---

3) Package & Run with Docker

Dockerfile (general-purpose)

```
# Use the smallest base that fits your runtime
FROM debian:stable-slim

# Install runtime deps (example: Java + unzip)
RUN apt-get update && apt-get install -y --no-install-recommends \
   openjdk-17-jre-headless unzip ca-certificates curl \
 && rm -rf /var/lib/apt/lists/*

WORKDIR /app
# Copy zip and expand
COPY myapp.zip .
RUN unzip myapp.zip -d /app && rm myapp.zip

# If you have a start script, make it executable
# RUN chmod +x bin/start.sh

# Expose port (change if needed)
```

```
EXPOSE 8080

# Start command (adjust for your app)
CMD ["java", "-jar", "app/myapp.jar"]
```

Build & Run

```
docker build -t myapp:zip .
docker run -d --name myapp -p 8080:8080 myapp:zip
```

For Node/Python, install the right base and set CMD accordingly.

---

4) Deploy to Kubernetes (via your Docker image)

1. **Push image** to a registry:

```
# Example using Docker Hub
docker tag myapp:zip <dockerhub-username>/myapp:zip
docker push <dockerhub-username>/myapp:zip
```

2. **Create Deployment & Service**

```
# k8s-deploy.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  selector:
    matchLabels: { app: myapp }
  template:
    metadata:
      labels: { app: myapp }
    spec:
      containers:
        - name: myapp
          image: <dockerhub-username>/myapp:zip
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet: { path: /healthz, port: 8080 }
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet: { path: /healthz, port: 8080 }
            initialDelaySeconds: 15
```

```
        periodSeconds: 20
---
apiVersion: v1
kind: Service
metadata:
  name: myapp
spec:
  selector: { app: myapp }
  ports:
    - port: 80
      targetPort: 8080
  type: LoadBalancer
kubectl apply -f k8s-deploy.yaml
kubectl get svc myapp
```

---

5) AWS

5.1 AWS Lambda (direct .zip)

Structure your zip as Lambda expects (handler file + dependencies). Then:

```
aws lambda create-function \
  --function-name myapp-fn \
  --runtime python3.11 \
  --role arn:aws:iam::<acct-id>:role/<lambda-exec-role> \
  --handler handler.lambda_handler \
  --zip-file fileb://myapp.zip

# Update code later:
aws lambda update-function-code \
  --function-name myapp-fn \
  --zip-file fileb://myapp.zip
```

5.2 Elastic Beanstalk (direct .zip)

Zip must contain app + optional Procfile.

```
eb init -p java-17 myapp-eb --region ap-south-1
eb create myapp-env
# Deploy zip
aws elasticbeanstalk create-application-version \
  --application-name myapp-eb \
  --version-label v1 \
  --source-bundle S3Bucket=my-bucket,S3Key=myapp.zip
aws elasticbeanstalk update-environment \
  --environment-name myapp-env \
  --version-label v1
```

5.3 CodeDeploy (direct .zip with appspec.yml)

Ensure your zip has **appspec.yml** + scripts.

appspec.yml (EC2/On-Prem example)

```
version: 0.0
os: linux
files:
  - source: /
    destination: /opt/myapp
hooks:
  ApplicationStop:
    - location: scripts/stop.sh
      timeout: 60
  BeforeInstall:
    - location: scripts/before_install.sh
      timeout: 300
  AfterInstall:
    - location: scripts/after_install.sh
      timeout: 300
  ApplicationStart:
    - location: scripts/start.sh
      timeout: 60
  ValidateService:
    - location: scripts/health_check.sh
      timeout: 60
```

Push & Deploy

```
aws deploy create-application --application-name myapp
aws deploy create-deployment-group \
  --application-name myapp \
  --deployment-group-name myapp-dg \
  --service-role-arn arn:aws:iam::<acct-id>:role/CodeDeployRole \
  --deployment-config-name CodeDeployDefault.AllAtOnce \
  --ec2-tag-filters Key=Name,Value=myapp,Type=KEY_AND_VALUE

# Upload to S3 and deploy
aws s3 cp myapp.zip s3://my-codedeploy-artifacts/myapp.zip
aws deploy create-deployment \
  --application-name myapp \
  --deployment-group-name myapp-dg \
  --s3-location bucket=my-codedeploy-artifacts,key=myapp.zip,bundleType=zip
```

5.4 EC2 user-data (unzip & run)

```
#!/bin/bash
apt-get update -y && apt-get install -y unzip openjdk-17-jre-headless
```

```
cd /opt && mkdir -p myapp && cd myapp
aws s3 cp s3://my-bucket/myapp.zip .
unzip myapp.zip && rm myapp.zip
nohup java -jar app/myapp.jar > app.log 2>&1 &
```

5.5 S3 Static Website

```
aws s3 mb s3://my-static-site
aws s3 website s3://my-static-site --index-document index.html --error-document 404.html
unzip -o myapp.zip -d site
aws s3 sync site/ s3://my-static-site --delete
```

---

6) Azure

6.1 App Service (Zip Deploy)

```
az webapp up -n myapp-web -g my-rg -l centralindia -p F1
az webapp deployment source config-zip \
  --resource-group my-rg --name myapp-web \
  --src myapp.zip
```

6.2 Azure Functions (Zip Deploy)

```
az functionapp create -g my-rg -n my-func -s mystorageacct -c centralindia --consumption-plan-location centralindia --runtime python
az functionapp deployment source config-zip -g my-rg -n my-func --src myapp.zip
```

6.3 Azure VM (unzip & run)

```
sudo apt-get update && sudo apt-get install -y unzip openjdk-17-jre-headless
sudo mkdir -p /opt/myapp && sudo unzip myapp.zip -d /opt/myapp
cd /opt/myapp && nohup java -jar app/myapp.jar > app.log 2>&1 &
```

6.4 Static website via Azure Storage

```
# Enable static site
az storage blob service-properties update --account-name mystorageacct --static-website --index-document index.html --404-document 404.html
unzip -o myapp.zip -d site
az storage blob upload-batch -s site -d '$web' --account-name mystorageacct
```

---

7) GCP

7.1 Cloud Functions (direct .zip)

```
gcloud functions deploy my-func \
  --region=asia-south1 \
```

```
  --runtime=python311 \
  --trigger-http \
  --source=myapp.zip
```

7.2 Cloud Run (via Docker image)

```
# Build and push using gcloud (Artifact Registry)
gcloud builds submit --tag asia-south1-docker.pkg.dev/<PROJECT_ID>/myrepo/myapp:zip .
gcloud run deploy myapp --image asia-south1-docker.pkg.dev/<PROJECT_ID>/myrepo/myapp:zip -
-region=asia-south1 --platform=managed --allow-unauthenticated
```

7.3 App Engine (source + app.yaml)

```
# app.yaml (example for Java via custom runtime)
runtime: java17
entrypoint: java -jar app/myapp.jar
# Unzip and include app.yaml in the root folder before deploy
unzip -o myapp.zip -d appengine-src
cp app.yaml appengine-src/
cd appengine-src && gcloud app deploy
```

7.4 Cloud Storage Static Website (behind Load Balancer)

```
unzip -o myapp.zip -d site
gsutil mb -l asia-south1 gs://my-static-site
gsutil -m rsync -r site gs://my-static-site
# (Optional) set up HTTPS LB pointing to the bucket for production use
```

---

8) Optional: GitHub Actions to ship your .zip

8.1 Deploy to Azure App Service

```
name: Deploy to Azure App Service
on: [push]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Upload zip
        uses: azure/webapps-deploy@v2
        with:
          app-name: myapp-web
          publish-profile: ${{ secrets.AZURE_PUBLISH_PROFILE }}
          package: myapp.zip
```

8.2 Deploy to AWS CodeDeploy

```
name: Deploy to CodeDeploy
on: [push]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Zip bundle
        run: zip -r myapp.zip .
      - name: Upload to S3
        run: aws s3 cp myapp.zip s3://my-codedeploy-artifacts/myapp.zip
      - name: Create deployment
        run: |
          aws deploy create-deployment \
            --application-name myapp \
            --deployment-group-name myapp-dg \
            --s3-location bucket=my-codedeploy-artifacts,key=myapp.zip,bundleType=zip
```

---

9) Troubleshooting Cheatsheet

- **Port not reachable**: Expose/map correct port (EXPOSE in Docker, Service targetPort in K8s, NSG/Security Group rules in cloud).
- **Startup fails**: Verify CMD/entrypoint; run container interactively: docker run -it --rm myapp:zip sh.
- **Permissions**: Ensure scripts are executable: chmod +x bin/*.sh.
- **Wrong path after unzip**: Check where files actually extracted; adjust Nginx root or app entrypoint.
- **Missing dependencies**: Install runtime packages in Dockerfile or VM.
- **Health probes failing**: Implement /healthz and increase initial delays.
- **Lambda/Functions size limit**: Prune dev deps, use layers.

---

10) Quick Decision Matrix

Target	Zip Direct?	Recommended Path
Web server	No	Unzip to document root
App server (Java)	Yes (WAR)	Package WAR, drop in webapps/
Docker	No	Unzip in Dockerfile and run
Kubernetes	No	Build Docker image, deploy YAML
AWS Lambda	Yes	Upload zip

AWS EB	Yes	Upload zip as version
AWS CodeDeploy	Yes	Zip + appspec.yml
AWS EC2	No	User-data or SSH unzip & run
AWS S3 static	N/A	Unzip locally, s3 sync
Azure App Service	Yes	ZipDeploy
Azure Functions	Yes	ZipDeploy
Azure VM	No	Unzip & run
GCP Cloud Functions	Yes	Deploy zip
GCP Cloud Run	No	Docker image
GCP App Engine	No	Source + app.yaml
GCS static	N/A	Unzip locally, gsutil rsync

Final Notes

- Keep **one canonical start command** and a **consistent directory layout** inside your zip to make these flows predictable.
- For production: add **logging**, **metrics**, and **CI/CD** to automate packaging and deployments.
- If you share:
  - your runtime (Java/Node/Python/etc.),
  - your start command,
  - and your target platform,

    I can tailor the exact commands to your stack in seconds.